

Machine learning for time-series

Machine learning is the study of algorithms that improve through experience. These algorithms or models can make systematic, repeatable, validated decisions based on data.

The main machine learning problems with time-series:

- Forecasting,
- Regression,
- Classification,
- Segmentation,
- Anomaly detection.

Machine learning for time-series

Making predictions about the future is called **forecasting**. Forecasting can be based only on the time-series values itself or on other variables.

For example, we could be forecasting global temperatures based on the data of the last 100 years, or we could be forecasting the economic wellbeing of a nation.

Machine learning for time-series

Regression is an umbrella term for statistical approaches for finding relationships between independent variables (features) and dependent variables (targets).

For instance, we could be predicting the exact temperature based on the release of carbon dioxide and methane.

Machine learning for time-series

When the problem is assigning labels to a time-series (or a part of it), this is called **classification**.

The main difference to regression is that the prediction is categorical rather than continuous.

Classification can be binary, when there are precisely two classes, or multi-class, when there are more categories.

An example would be detecting eye movements or epilepsy in EEG signals.

Machine learning for time-series

Segmentation, or **clustering**, is the process of grouping parts of the time-series into clusters (or segments) of different regimes, behaviors, or baselines.

An example would be different activity levels in brain waves.

Machine learning for time-series

Anomaly detection, also known as **outlier detection**, is the task of identifying events that are rare or outside the norm.

Example: An outage of the electricity grid detectable in a sudden drop of the voltage.

Techniques in anomaly detection can range from simple thresholding or statistics to a set of rules, to pattern-based approaches based on the time-series distribution.

Machine learning for time-series

In **supervised** learning, the features are mapped to outcomes $y \in Y$ in a process called **prediction**.

Parameters are estimated from labeled observations. We need to have the outcome available for each observation as the **target** column.

The machine learning algorithm finds a mapping from X to Y .

$$f: X \rightarrow Y$$

The function $f \in F$ is just one possible mapping or model of the input distribution X to the output distribution Y .

Machine learning for time-series

In time-series **forecasting**, the historical values are extrapolated into the future. The only features are the past values.

For example, we could be estimating the calls into a call center over the next month based on calls during the last 2 years.

The mapping function, f , predicts an outcome $f(X) \rightarrow \tilde{Y}$.

Each function is specified by a set of parameters, and the optimization results in a set of parameters that minimizes the mismatch between Y and $\tilde{Y}$.

Machine learning for time-series

To ensure that parameters estimated on a dataset of limited size are still valid for more data, we must go through a validation that makes sure that the quality holds up.

For validation, we usually **split the dataset into at least two parts**, the **training set** and the **test set**.

We estimate parameters on a training set, then run the model on the test set to get an idea of the quality of the model on unseen data points.

In machine learning, the process of testing a model on data it hasn't seen in training is called **cross-validation** (sometimes, **out-of-sample testing**).

Error metrics for time-series

If $f(x_t)$ is a prediction of the model for time step t , and the actual target value is y_t , the **forecast error (residual)** is the difference between the actual values of the target and the values our model predicts:

$$e_t = y_t - f(x_t)$$

This compares the actual target Y to the predicted targets $\hat{Y} = f(X)$. The **sum of squares of the residuals** (SS, also **residual sum of squares**)

$$SS = \sum_{t=1}^N (y_t - f(x_t))^2 = \sum_{t=1}^N e_t^2$$

is used to measure the performance of predictions over a time-series.

Error metrics for time-series

In practice, the most popular error metrics are the mean squared error (MSE), mean absolute error (MAE), and the root mean squared error (RMSE:

Metric Name	Definition
Mean squared error	$\text{MSE} = \frac{1}{N} \sum_{t=1}^N e_t^2$
Mean absolute error	$\text{MAE} = \frac{1}{N} \sum_{t=1}^N e_t $
Root mean squared error	$\text{RMSE} = \sqrt{\text{MSE}}$

Error metrics for time-series

There are many more error metrics in time-series modeling:

Metric Name	Definition
Median absolute error	$\text{MdAE} = \text{median}(e _t)$
Mean absolute percentage error	$\text{MAPE} = \frac{1}{N} \sum_{t=1}^N \frac{ e_t }{ y_t }$
Symmetric mean absolute percentage error	$\text{SMAPE} = \frac{1}{N} \sum_{t=1}^N \frac{ e_t }{(y_t + f(x_t))/2}$
Normalized mean squared error	$\text{NMSE} = \frac{\text{MSE}}{\sigma^2}$

Box-Jenkins Method in Time Series Forecasting

Autoregressive Integrated Moving Average Model

The Autoregressive Integrated Moving Average Model, or ARIMA for short is a standard statistical model for time series forecast and analysis. This acronym is descriptive, capturing the key aspects of the model itself:

AR: *Autoregression*. A model that uses the dependent relationship between an observation and some number of lagged observations.

I: *Integrated*. The use of differencing of raw observations (i.e. subtracting an observation from an observation at the previous time step) in order to make the time series stationary.

MA: *Moving Average*. A model that uses the dependency between an observation and residual errors from a moving average model applied to lagged observations.

Autoregressive Integrated Moving Average Model

Each of these components are explicitly specified in the model as a parameter. A standard notation is used of $ARIMA(p,d,q)$ where the parameters are substituted with integer values to quickly indicate the specific ARIMA model being used. The parameters of the ARIMA model are defined as follows:

p: The number of lag observations included in the model, also called the lag order.

d: The number of times that the raw observations are differenced, also called the degree of differencing.

q: The size of the moving average window, also called the order of moving average.

Configuring AR and MA

Two plots can be used to choose the p and q parameters of the ARMA or ARIMA:

Autocorrelation Function (ACF). The plot summarizes the correlation of an observation with lag values. The x-axis shows the lag and the y-axis shows the correlation coefficient between -1 and 1.

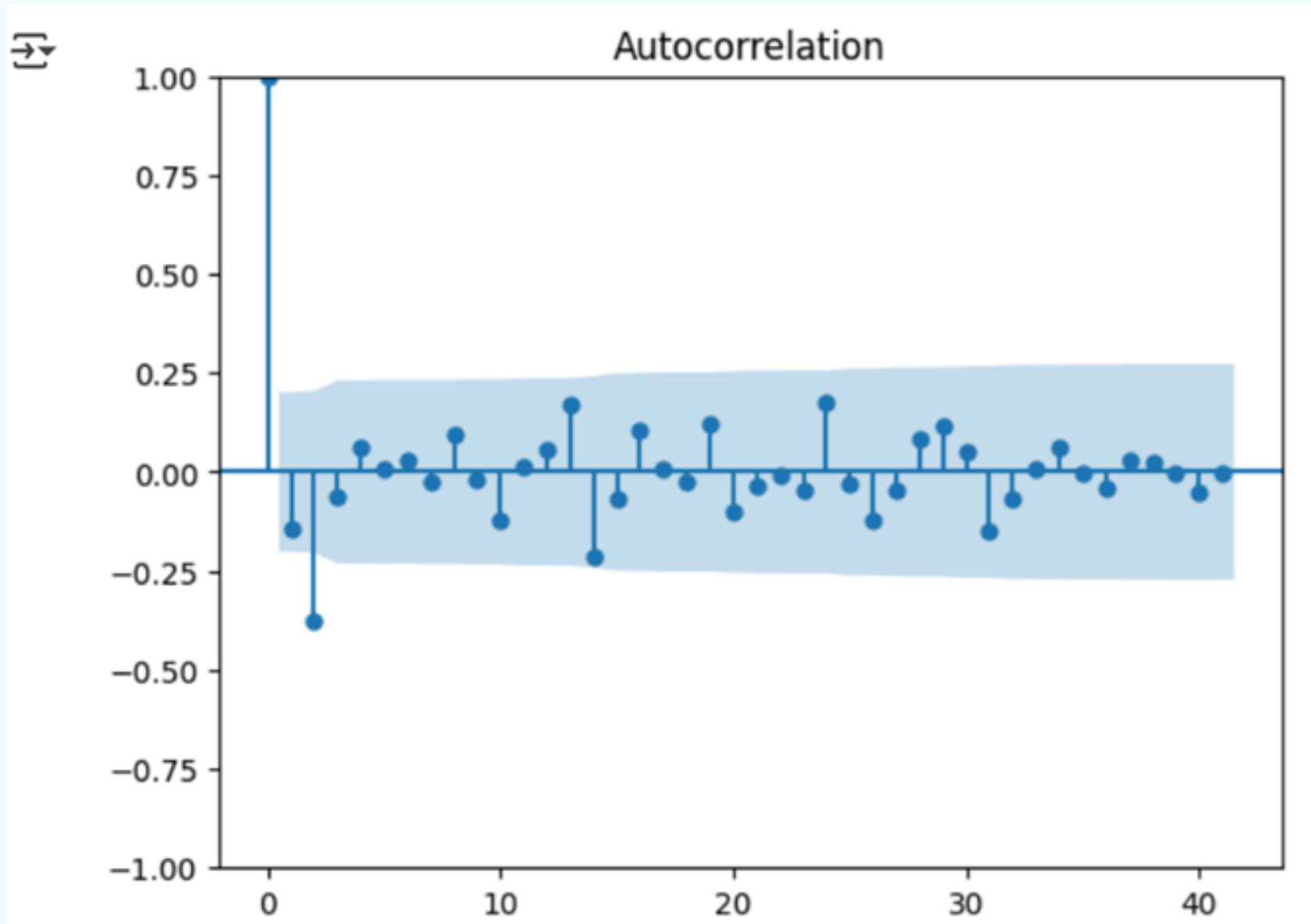
Partial Autocorrelation Function (PACF). The plot summarizes the correlations for an observation with lag values that is not accounted for by prior lagged observations.

Both plots are drawn as bar charts showing the 95% and 99% confidence intervals as horizontal lines. Bars that cross these confidence intervals are more significant and worth noting. Some useful patterns you may observe on these plots are:

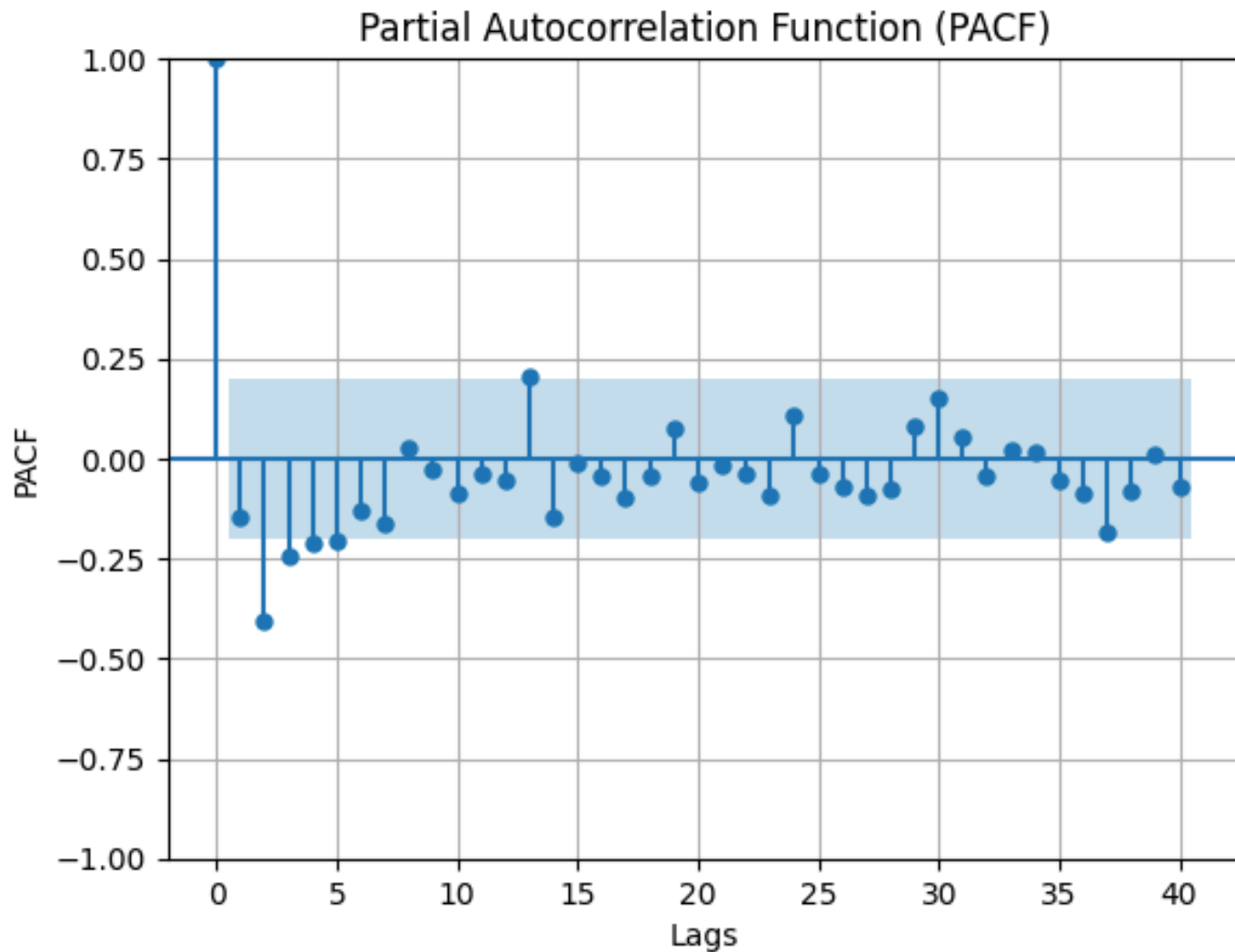
The model is AR if the ACF trails off after a lag and has a hard cut-off in the PACF after a lag. **This lag is taken as the value for p .**

The model is MA if the PACF trails off after a lag and has a hard cut-off in the ACF after the lag. **This lag value is taken as the value for q .**

Configuring AR and MA



Configuring AR and MA



Configuring ARIMA

Stationarity checking can be used to choose the d of the ARIMA:

Take the parameter $d = 0$ if the main series S_t is stationary.

Take the parameter $d = 1$ if the main series S_t is non-stationary, but the first-order difference series ΔS_t is stationary.

Take the parameter $d = 2$ if the main series S_t and the first-order difference series ΔS_t are non-stationary, but the second-order difference series $\Delta^2 S_t$ is stationary ...

The parameter d in ARIMA would typically be 2 or less.

Stationarity checking

Augmented Dickey-Fuller test

This is a statistical test that is dedicatedly built to test whether univariate time series data is stationary or not. Let's define the null and alternate hypotheses,

- **Ho (Null Hypothesis):** The time series data is non-stationary
 - **H1 (alternate Hypothesis):** The time series data is stationary
- Assume **alpha = 0.05**, meaning (95% confidence). The test results are interpreted with a p-value if $p > 0.05$ fails to reject the null hypothesis, else if $p \leq 0.05$ reject the null hypothesis.

Stationarity checking

```
▶ from google.colab import files  
import pandas as pd  
import io  
uploaded = files.upload()
```



Choose Files SolieuThuTSA001.csv

SolieuThuTSA001.csv(text/csv) - 3415 bytes, last modified: 9/21/2025 - 100% done
Saving SolieuThuTSA001.csv to SolieuThuTSA001.csv

```
▶ data = pd.read_csv('SolieuThuTSA001.csv')
```

```
CPI_rate = data['CPI_rate']
```

```
from statsmodels.tsa.stattools import adfuller
```

Stationarity checking

```
▶ result = adfuller(CPI_rate)

# Extract and display results
print("ADF Statistic:", result[0])
print("p-value:", result[1])
print("Critical Values:")
for key, value in result[4].items():
    print(f"    {key}: {value}")

# Interpretation
if result[1] <= 0.05:
    print("The time series is stationary (reject the null hypothesis).")
else:
    print("The time series is not stationary (fail to reject the null hypothesis).")
```

Stationarity checking



ADF Statistic: -7.446385614920216

p-value: 5.823387502646691e-11

Critical Values:

1%: -3.50434289821397

5%: -2.8938659630479413

10%: -2.5840147047458037

The time series is stationary (reject the null hypothesis).

Estimating ARIMA model 1

```
data = pd.read_csv('SolieuThuTSA001.csv')  
series = data['CPI_rate']
```

```
pip install statsmodels pandas_datereader
```

```
from statsmodels.tsa.arima_model import ARIMA
```

```
# Prepare data
```

```
X = series.values
```

```
X = X.astype('float32')
```

```
# Fit model
```

```
model = ARIMA(X, order=(1, 1, 1))
```

```
model_fit = model.fit()
```


Estimating ARIMA model 1

```
# summary of fit model
print(model_fit.summary())
# line plot of residuals
residuals = DataFrame(model_fit.resid)
residuals.plot()
pyplot.show()
# density plot of residuals
residuals.plot(kind='kde')
pyplot.show()
# summary stats of residuals
print(residuals.describe())
```

Estimating ARIMA model 1



SARIMAX Results

```

=====
Dep. Variable:                y      No. Observations:                96
Model:                ARIMA(1, 1, 1)  Log Likelihood                -94.105
Date:                Wed, 01 Oct 2025  AIC                194.210
Time:                15:08:29         BIC                201.871
Sample:                0              HQIC                197.306
                                - 96
Covariance Type:                opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.1403	0.106	-1.320	0.187	-0.348	0.068
ma.L1	-0.9997	8.208	-0.122	0.903	-17.088	15.089
sigma2	0.4036	3.299	0.122	0.903	-6.062	6.869

```

=====
Ljung-Box (L1) (Q):                0.08  Jarque-Bera (JB):                0.31
Prob(Q):                0.78  Prob(JB):                0.86
Heteroskedasticity (H):                1.65  Skew:                0.02
Prob(H) (two-sided):                0.16  Kurtosis:                3.28
=====

```

Estimating ARIMA model 2

```
# Fit model
model = ARIMA(X, order=(2, 0, 1))
model_fit = model.fit()
# summary of fit model
print(model_fit.summary())
# line plot of residuals
residuals = DataFrame(model_fit.resid)
residuals.plot()
pyplot.show()
# density plot of residuals
residuals.plot(kind='kde')
pyplot.show()
# summary stats of residuals
print(residuals.describe())
```

Estimating ARIMA model 2



SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          96
Model:                ARIMA(2, 0, 1)  Log Likelihood      -74.482
Date:                Wed, 01 Oct 2025  AIC                158.963
Time:                15:12:07    BIC                171.785
Sample:                0      HQIC                164.146
                        - 96
Covariance Type:          opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	0.0107	0.010	1.095	0.273	-0.008	0.030
ar.L1	0.3939	0.116	3.408	0.001	0.167	0.620
ar.L2	-0.3204	0.121	-2.639	0.008	-0.558	-0.082
ma.L1	-0.8574	0.076	-11.350	0.000	-1.005	-0.709
sigma2	0.2723	0.031	8.771	0.000	0.211	0.333

```
=====
Ljung-Box (L1) (Q):          0.00  Jarque-Bera (JB):          9.86
Prob(Q):                    0.95  Prob(JB):              0.01
Heteroskedasticity (H):      1.62  Skew:                  0.27
Prob(H) (two-sided):        0.18  Kurtosis:              4.47
=====
```

Estimating ARIMA model 3

```
# Fit model
model = ARIMA(X, order=(2, 1, 1))
model_fit = model.fit()
# summary of fit model
print(model_fit.summary())
# line plot of residuals
residuals = DataFrame(model_fit.resid)
residuals.plot()
pyplot.show()
# density plot of residuals
residuals.plot(kind='kde')
pyplot.show()
# summary stats of residuals
print(residuals.describe())
```

Estimating ARIMA model 3



SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          96
Model:                ARIMA(2, 1, 1)  Log Likelihood      -85.591
Date:                Wed, 01 Oct 2025  AIC                179.182
Time:                15:16:08    BIC                189.398
Sample:              0      HQIC                183.310
                        - 96
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.2014	0.093	-2.166	0.030	-0.384	-0.019
ar.L2	-0.4106	0.102	-4.008	0.000	-0.611	-0.210
ma.L1	-0.9998	12.761	-0.078	0.938	-26.011	24.011
sigma2	0.3334	4.241	0.079	0.937	-7.979	8.646

```
=====
Ljung-Box (L1) (Q):      0.55  Jarque-Bera (JB):      4.59
Prob(Q):                0.46  Prob(JB):            0.10
Heteroskedasticity (H):  1.80  Skew:                0.17
Prob(H) (two-sided):    0.10  Kurtosis:            4.02
=====
```

Model selection

When we have different models, how do we know which one should we use? Model selection is the methodology for deciding between competing models.

ARMA and ARIMA models are usually estimated with the **maximum-likelihood estimation (MLE)**.

One of the most commonly used model selection criteria for the maximum-likelihood method is the **Akaike information criterion (AIC)**.

Akaike information criterion

AIC takes the log-likelihood l from the maximum-likelihood method and the number of parameters k in the model.

$$AIC = 2k - 2l$$

For an ARIMA model, it is written more specifically:

$$AIC = 2(p + q) - 2l$$

A model is considered to have **higher quality** if it has a **lower AIC**.

Bayesian Information Criterion

The **Bayesian Information Criterion (BIC)** looks very much like AIC. It additionally takes N , the sample size in the dataset:

$$BIC = k \cdot \ln N - 2l$$

For an ARIMA model, it is written more specifically:

$$BIC = (p + q) \cdot \ln N - 2l$$

According to BIC, we want a model with **few parameters** and **high log-likelihood**, but also a **small number of training examples**.

A model is considered to have **higher quality** if it has a **lower BIC**.

Hannan–Quinn Information Criterion

The **Hannan–Quinn information criterion (HQIC)** is an alternative to AIC and BIC. It is given as

$$HQIC = 2k \cdot \ln(\ln(N)) - 2l$$

For an ARIMA model, it is written more specifically:

$$BIC = (p + q) \cdot \ln(\ln(N)) - 2l$$

A model is considered to have **higher quality** if it has a **lower HQIC**.

Bayesian Information Criterion

The **Bayesian Information Criterion (BIC)** looks very much like AIC. It additionally takes N , the sample size in the dataset:

$$BIC = k \cdot \ln N - 2l$$

For an ARIMA model, it is written more specifically:

$$BIC = (p + q) \cdot \ln N - 2l$$

According to BIC, we want a model with **few parameters** and **high log-likelihood**, but also a **small number of training examples**.

A model is considered to have **higher quality** if it has a **lower BIC**.

Stability of ARIMA model

Stability in regression models refers to the consistency of the model's coefficients across different time periods or data sets. When coefficients change significantly, it indicates a structural break, which can lead to unreliable predictions.

The stability can be assessed by examining the confidence intervals of the model coefficients. If the CI of a coefficient contains 0, the estimate of this coefficient will easily change from a negative value to a positive value or vice versa. Then the model is unstable, coefficients with CI containing 0 should be removed from the model.

ARIMA model 1



SARIMAX Results

```

=====
Dep. Variable:          y      No. Observations:          96
Model:                ARIMA(1, 1, 1)      Log Likelihood      -94.105
Date:                Wed, 01 Oct 2025      AIC                  194.210
Time:                15:08:29      BIC                  201.871
Sample:                0      HQIC                  197.306
                        - 96
Covariance Type:          opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.1403	0.106	-1.320	0.187	-0.348	0.068
ma.L1	-0.9997	8.208	-0.122	0.903	-17.088	15.089
sigma2	0.4036	3.299	0.122	0.903	-6.062	6.869

```

=====
Ljung-Box (L1) (Q):          0.08      Jarque-Bera (JB):          0.31
Prob(Q):                    0.78      Prob(JB):              0.86
Heteroskedasticity (H):      1.65      Skew:                  0.02
Prob(H) (two-sided):        0.16      Kurtosis:              3.28
=====

```

Parameters ar.L1 and ma.L1 should be removed $\square$ $p = 0$ and $q = 0$. The model fails.

ARIMA model 3

↕

SARIMAX Results						
=====						
Dep. Variable:	y	No. Observations:	96			
Model:	ARIMA(2, 1, 1)	Log Likelihood	-85.591			
Date:	Wed, 01 Oct 2025	AIC	179.182			
Time:	15:16:08	BIC	189.398			
Sample:	0	HQIC	183.310			
	- 96					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ar.L1	-0.2014	0.093	-2.166	0.030	-0.384	-0.019
ar.L2	-0.4106	0.102	-4.008	0.000	-0.611	-0.210
ma.L1	-0.9998	12.761	-0.078	0.938	-26.011	24.011
sigma2	0.3334	4.241	0.079	0.937	-7.979	8.646
=====						
Ljung-Box (L1) (Q):	0.55	Jarque-Bera (JB):	4.59			
Prob(Q):	0.46	Prob(JB):	0.10			
Heteroskedasticity (H):	1.80	Skew:	0.17			
Prob(H) (two-sided):	0.10	Kurtosis:	4.02			
=====						

Parameter ma.L1 should be removed q = 0.

ARIMA model 2



SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          96
Model:                ARIMA(2, 0, 1)  Log Likelihood      -74.482
Date:                Wed, 01 Oct 2025  AIC                158.963
Time:                15:12:07    BIC                171.785
Sample:              0      HQIC                164.146
                        - 96
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	0.0107	0.010	1.095	0.273	-0.008	0.030
ar.L1	0.3939	0.116	3.408	0.001	0.167	0.620
ar.L2	-0.3204	0.121	-2.639	0.008	-0.558	-0.082
ma.L1	-0.8574	0.076	-11.350	0.000	-1.005	-0.709
sigma2	0.2723	0.031	8.771	0.000	0.211	0.333

```
=====
Ljung-Box (L1) (Q):          0.00  Jarque-Bera (JB):          9.86
Prob(Q):                    0.95  Prob(JB):              0.01
Heteroskedasticity (H):      1.62  Skew:                  0.27
Prob(H) (two-sided):        0.18  Kurtosis:              4.47
=====
```

Using “loop” in modelling

```
for p in range(1, 9, 1):  
    { model = ARIMA(X, order=(p, 0, 1))  
      model_fit = model.fit()  
      # summary of fit model  
      print(model_fit.summary())  
      # line plot of residuals  
      residuals = DataFrame(model_fit.resid)  
      residuals.plot()  
      pyplot.show()  
      # density plot of residuals  
      residuals.plot(kind='kde')  
      pyplot.show()  
      # summary stats of residuals  
      print(residuals.describe())  
    }
```


Forecasting ARIMA model

Mean values Forecasting

```
from matplotlib import pyplot as plt
```

```
STEPS = 20
```

```
forecasts_df = model_fit.get_forecast  
(steps=STEPS).summary_frame()
```

```
df1 = series
```

```
ax = df1.plot(figsize=(12, 6))
```

```
plt.ylabel('CPI_rate')
```

```
forecasts_df['mean'].plot(style='k--')
```

```
ax.fill_between(
```

```
forecasts_df.index,
```

```
forecasts_df['mean_ci_lower'],
```

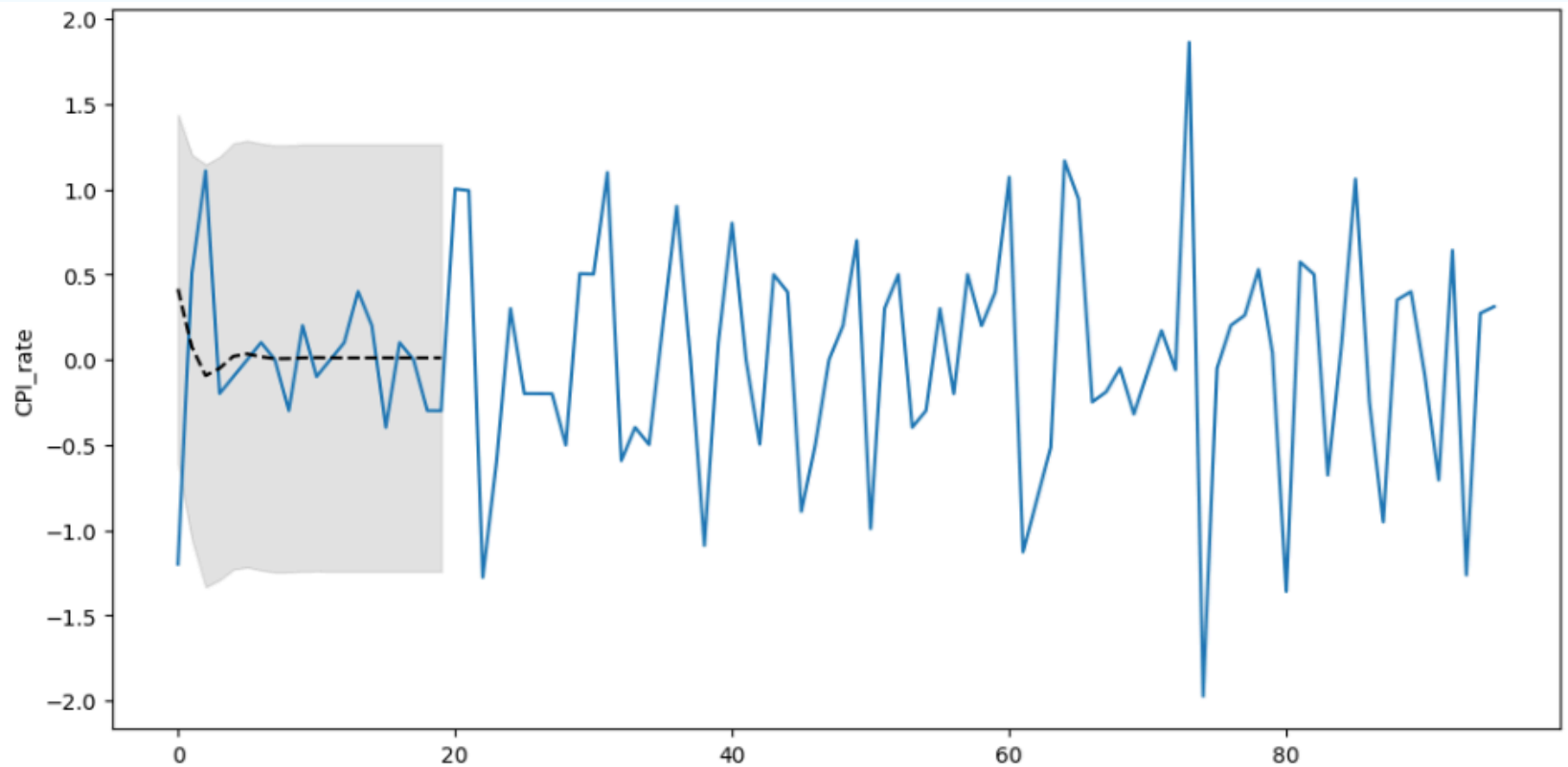
```
forecasts_df['mean_ci_upper'],
```

```
color='k',
```

```
alpha=0.1
```

```
)
```

Forecasting ARIMA model



Forecasting ARIMA model

```
# Split data series to train part and test part
```

```
train = series[series.index <= 80]
```

```
test = series[series.index > 80]
```

```
plt.plot(train, color = "black")
```

```
plt.plot(test, color = "red")
```

```
plt.ylabel('CPI_rate')
```

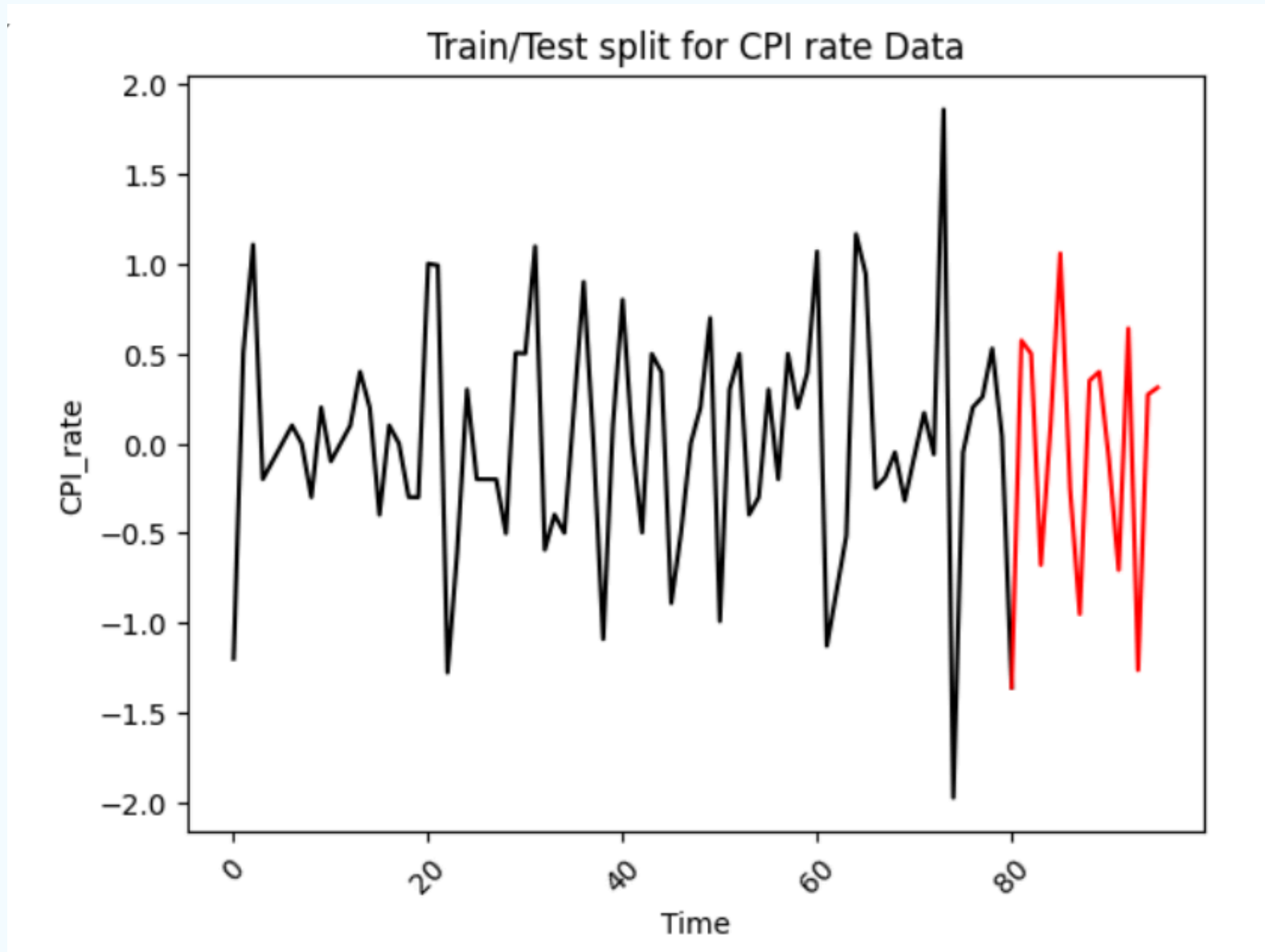
```
plt.xlabel('Time')
```

```
plt.xticks(rotation=45)
```

```
plt.title("Train/Test split for CPI rate Data")
```

```
plt.show()
```

Forecasting ARIMA model



Forecasting ARIMA model

```
from statsmodels.tsa.arima.model import ARIMA
```

```
ARIMAmode = ARIMA(y, order = (2, 0, 1))
```

```
ARIMAmode = ARIMAmode.fit()
```

```
y_pred = ARIMAmode.get_forecast(len(test.index))
```

```
y_pred_df = y_pred.conf_int(alpha = 0.05)
```

```
y_pred_df["Predictions"] = ARIMAmode.predict(start =  
y_pred_df.index[0], end = y_pred_df.index[-1])
```

```
y_pred_df.index = test.index
```

```
y_pred_out = y_pred_df["Predictions"]
```

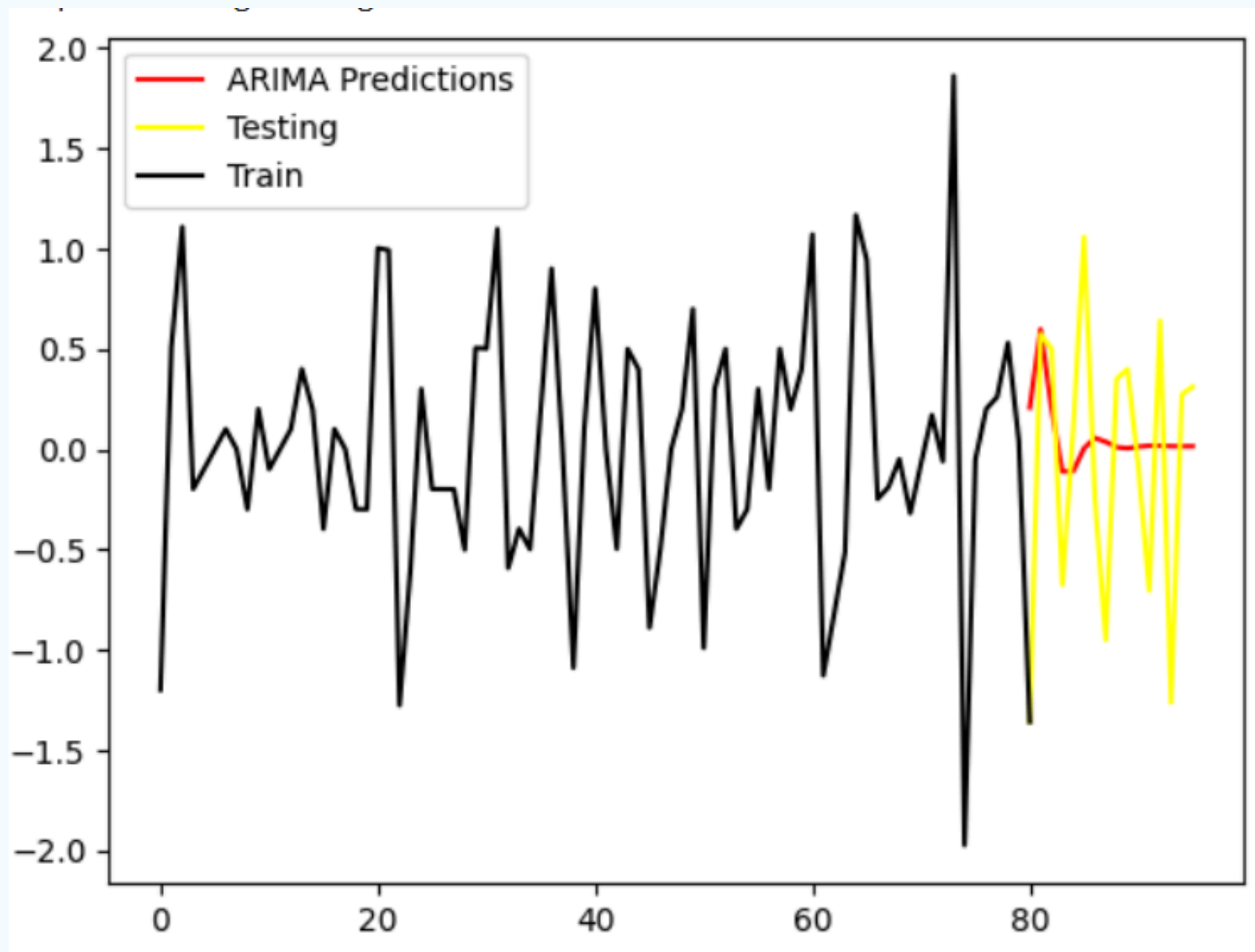
```
plt.plot(y_pred_out, color='red', label = 'ARIMA Predictions')
```

```
plt.plot(test, color='Yellow', label = 'Testing')
```

```
plt.plot(train, color='black', label = 'Train')
```

```
plt.legend()
```

Forecasting ARIMA model



Forecasting ARIMA model

```
# Evaluate the accuracy of forecasting

import numpy as np
from sklearn.metrics import mean_squared_error

arma_rmse =
np.sqrt(mean_squared_error(test.values,
y_pred_df["Predictions"]))
print("RMSE: ",arma_rmse)
```



```
RMSE: 0.7099876953323595
```

Seasonal ARIMA (SARIMA) Model

Seasonal ARIMA captures historical values, shock events and seasonality.

The general notation for a SARIMA model is expressed as:

$SARIMA(p,d,q)(P,D,Q, m)$. Where:

- p : Order of the non-seasonal autoregressive part.
- d : Degree of differencing (non-seasonal).
- q : Order of the non-seasonal moving average part.
- P : Order of the seasonal autoregressive part.
- D : Degree of seasonal differencing.
- Q : Order of the seasonal moving average part.
- m : Number of periods in each season

(e.g., 12 for monthly data with yearly seasonality).

Seasonal ARIMA (SARIMA) Model

We can define a SARIMA model using the SARIMAX class:

```
from statsmodels.tsa.statespace.sarimax  
import SARIMAX
```

Seasonal ARIMA (SARIMA) Model

```
SARIMAXmodel = SARIMAX(y, order = (2, 0, 1), seasonal_order=  
(2,0,1,12))
```

```
SARIMAXmodel = SARIMAXmodel.fit()
```

```
y_pred = SARIMAXmodel.get_forecast(len(test.index))
```

```
y_pred_df = y_pred.conf_int(alpha = 0.05)
```

```
y_pred_df["Predictions"] = SARIMAXmodel.predict(start =  
y_pred_df.index[0], end = y_pred_df.index[-1])
```

```
y_pred_df.index = test.index
```

```
y_pred_out = y_pred_df["Predictions"]
```

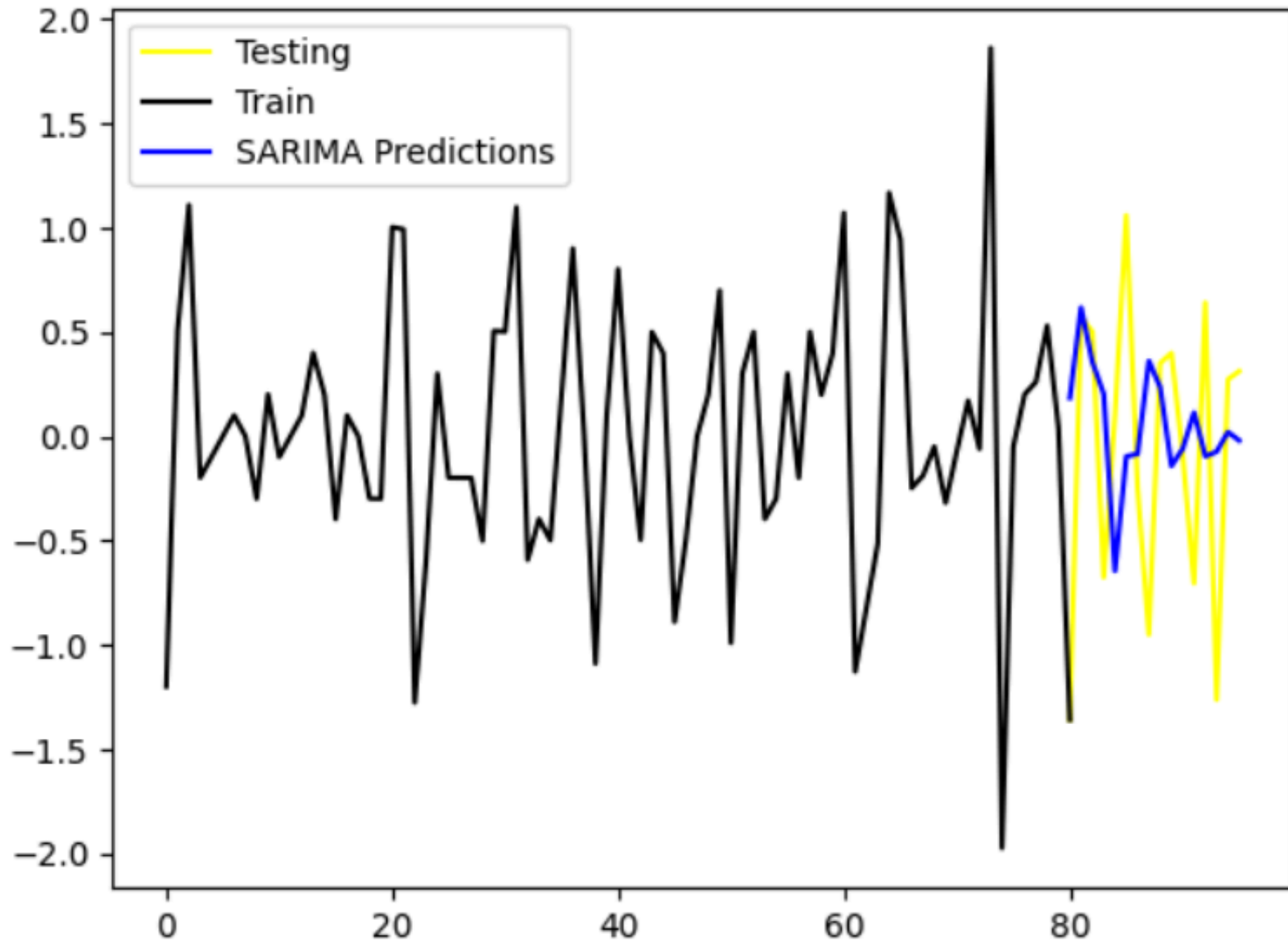
```
plt.plot(test, color='Yellow', label = 'Testing')
```

```
plt.plot(train, color='black', label = 'Train')
```

```
plt.plot(y_pred_out, color='Blue', label = 'SARIMA Predictions')
```

```
plt.legend()
```

Seasonal ARIMA (SARIMA) Model



Seasonal ARIMA (SARIMA) Model

```
import numpy as np
from sklearn.metrics import mean_squared_error

arma_rmse = np.sqrt(mean_squared_error(test.values,
y_pred_df["Predictions"]))
print("RMSE: ", arma_rmse)
```



```
RMSE: 0.7877833510873162
```

THANK YOU